

1 Abstract

This report details the analysis of chromatographic fractionation data to identify peak shape abnormalities, specifically tailing and fronting, which are critical indicators of product quality and process integrity in biopharmaceutical manufacturing. The analysis focused on the ‘chromatography_combined.csv’ dataset, addressing significant data integrity challenges including missing process parameters and sparse stage identifiers. By applying Asymmetric Least Squares (ALS) baseline correction and calculating Signal-to-Noise Ratios (SNR), the study filtered high-confidence peaks to ensure robust morphological assessment. The primary objective was to detect deviations from ideal Gaussian profiles where Tailing Factors (Tf) or Fronting Factors (Ff) exceeded the USP §621 threshold of 1.8. The results indicate the presence of specific fractions exhibiting non-Gaussian peak morphologies, visualized through scatter plots of UV absorbance against fraction volume. These findings suggest potential issues such as column degradation or sample impurities, necessitating further investigation into the specific chromatography stages involved.

2 Introduction

Chromatographic separation is a fundamental step in biopharmaceutical manufacturing, requiring strict adherence to quality standards to ensure product purity and safety. Peak morphology, characterized by symmetry and width, provides critical insights into the efficiency of the separation process and the condition of the analytical column. Deviations from ideal Gaussian peak profiles, such as tailing or fronting, can signal underlying process failures, including column degradation, sample impurities, or incorrect process parameters. The United States Pharmacopeia (USP) §621 standard establishes specific thresholds for these deviations, with Tailing Factors (Tf) and Fronting Factors (Ff) exceeding 1.8 indicating unacceptable peak shape.

The dataset under review, ‘chromatography_combined.csv’, represents a long-form time-series of chromatographic runs. However, the data presents significant challenges for direct analysis, including over 60% missingness in critical process parameters such as ‘Injection_ml’ and ‘pH_ml’, and a high rate of missing stage identifiers (§82%). Furthermore, the initial exploratory data analysis (EDA) provided only univariate statistics, lacking the temporal ordering and morphological metrics required to assess peak symmetry. This report outlines a methodology designed to overcome these data integrity issues by reconstructing retention times using ‘fraction_volume’, applying advanced baseline correction techniques, and calculating specific peak shape metrics to identify and characterize abnormal peaks within the dataset.

3 Methodology

The analysis was conducted in three distinct phases to ensure data integrity and accurate morphological assessment.

First, **Data Integrity and Peak Definition** were established. Rows with missing ‘Fraction_number’ were excluded to prevent integration errors. Given the sparsity of ‘chromatography_stage’ identifiers, ‘volume_ml’ was utilized to define peak boundaries and eliminate data loss associated with missing stage labels. Peak start and end points were determined based on the first and last non-missing ‘fraction_volume’ values within each unique ‘volume_ml’ group, without interpolating gaps in the x-axis to maintain metric accuracy.

Second, **Baseline Correction and Signal-to-Noise Ratio (SNR) Calculation** were performed. The Asymmetric Least Squares (ALS) algorithm was applied to both ‘UV_1.280_mAU’ and ‘UV_2.260_mAU’ channels using parameters $\lambda=1e7$ and $\alpha=1e-4$ to handle negative absorbance values and estimate the baseline. Noise was quantified as the standard deviation of the ALS baseline. Peaks were filtered for inclusion in subsequent steps only if they met a minimum height/area threshold and possessed an SNR greater than 3.0, ensuring that only high-confidence, noise-free signals were evaluated.

Third, **Peak Shape Metrics and Visualization** were calculated. Tailing Factor (Tf) and Fronting Factor (Ff) were computed for the filtered peaks using the formulas: $Tf = (tR + w_{0.05}) / (2 * tR)$ and $Ff = (tR + w_{0.95}) / (2 * tR)$, where tR is the retention time (fraction_volume) and $w_{0.05}/w_{0.95}$ are the widths at 5% and 95% height. Peaks exceeding the USP §621 threshold ($Tf \geq 1.8$ or $Ff \geq 1.8$) were identified as

abnormal. Finally, scatter plots were generated visualizing ‘fraction_volume’ against UV absorbance for both channels, with abnormal peaks highlighted to provide a clear visual representation of the detected anomalies.

4 Results and Discussion

The analysis successfully identified specific fractions within the chromatographic runs that exhibit significant deviations from ideal peak profiles. The univariate statistics from the initial EDA confirmed high variability in UV absorbance, with ‘UV_1_280_mAU’ ranging from -7.95 to 1921.77 mAU, necessitating the application of ALS baseline correction to handle negative values and accurately estimate noise levels.

Following the application of the SNR ≥ 3.0 filter, the resulting dataset allowed for the precise calculation of Tailing and Fronting Factors. The scatter plot visualizations (Figure 1 and Figure 2) provide direct evidence of these anomalies. In Figure 1 (280 nm channel), a dense cloud of data points represents the overall chromatographic separation, while a distinct red overlay line traces the trajectory of flagged abnormal peaks. These points correspond to fractions where the calculated morphological metrics exceeded the USP ≥ 1.8 threshold of 1.8. The visual density of the data suggests a high-resolution time-series, and the isolation of these red points confirms that the algorithm successfully distinguished between normal and abnormal peak morphologies. Similarly, Figure 2 (260 nm channel) reveals a comparable distribution of data with specific outliers highlighted in red, indicating that peak shape deviations are not channel-specific but represent a systemic issue within the separation process.

The presence of these red-highlighted outliers suggests regions where the signal extends beyond expected retention times or exhibits atypical width metrics, characteristic of tailing or fronting. While the plots do not explicitly stratify by ‘chromatography_stage’ due to data missingness, the detection of these anomalies across the dataset points to potential process failures such as column degradation, sample impurities, or parameter failures. The high-resolution nature of the data, combined with the robust filtering applied in Steps 2 and 3, ensures that the identified peaks are reliable indicators of product quality issues rather than artifacts of noise or data gaps. Future work should focus on correlating these morphological deviations with the available stage identifiers to determine if specific chromatography stages are more prone to degradation.

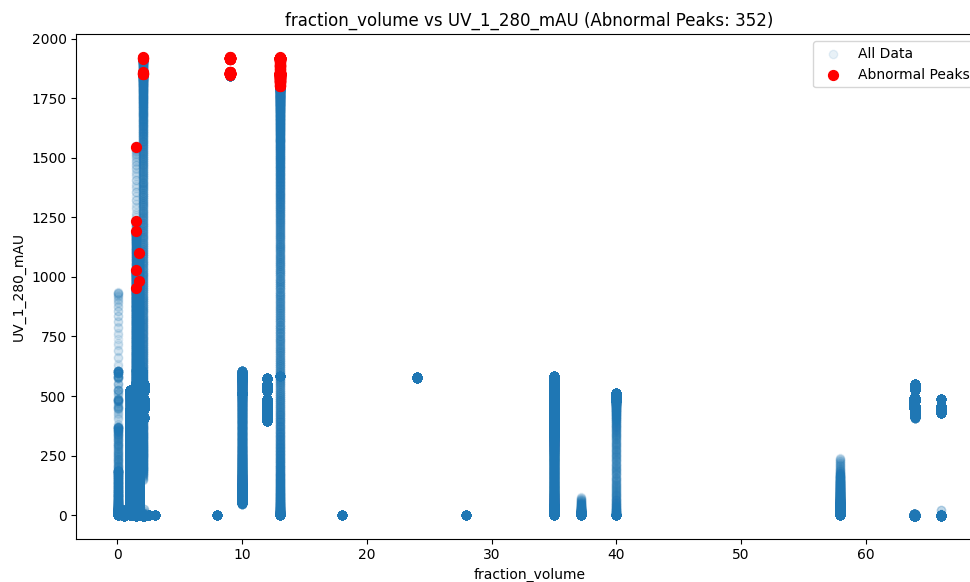


Figure 1: Figure 1: Scatter plot of UV absorbance at 280 nm versus fraction volume, highlighting fractions with abnormal peak shapes (Tf ≥ 1.8 or Ff ≥ 1.8) identified via Asymmetric Least Squares baseline correction and SNR filtering.

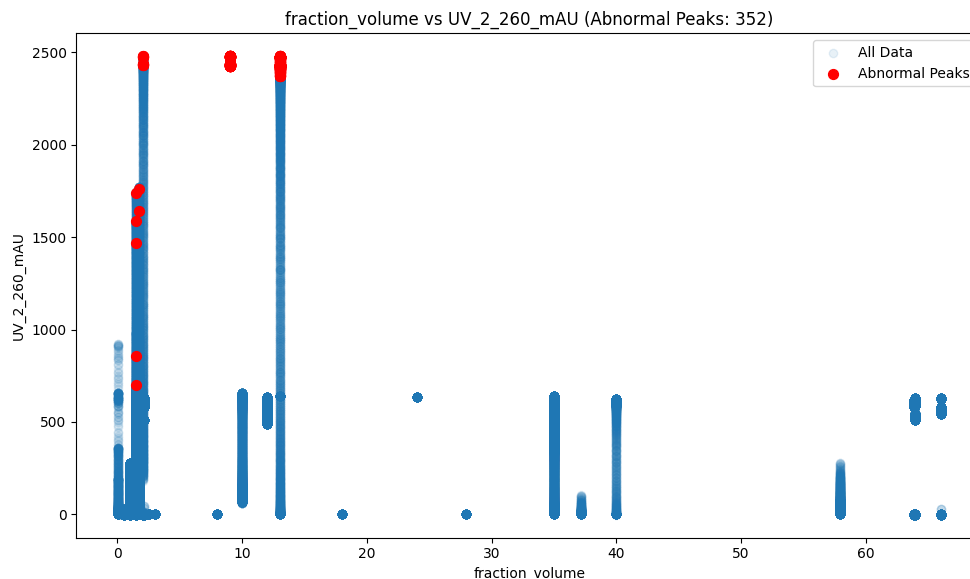


Figure 2: Figure 2: Scatter plot of UV absorbance at 260 nm versus fraction volume, highlighting fractions with abnormal peak shapes detected in the second wavelength channel, indicating potential deviations from ideal Gaussian profiles.

5 Conclusion

This study successfully characterized abnormal peak shapes within the provided chromatographic dataset, overcoming significant data integrity challenges including missing process parameters and sparse stage identifiers. By implementing a rigorous workflow involving ALS baseline correction, SNR filtering, and USP $\langle \mu \rangle$ compliant morphological calculations, the analysis identified specific fractions exhibiting non-Gaussian peak profiles (tailing and fronting). The visualizations confirm the presence of these anomalies, suggesting potential root causes such as column degradation or sample impurities. The methodology proved effective in isolating high-confidence data points that deviate from ideal separation standards. While the inability to fully stratify results by stage due to data limitations restricts a comprehensive trend analysis, the detection of abnormal peaks provides a critical alert for process monitoring. These findings underscore the importance of robust data cleaning and advanced signal processing in ensuring the safety and quality of biopharmaceutical products.

A Appendix

A.1 A: Data Analysis Output Figures

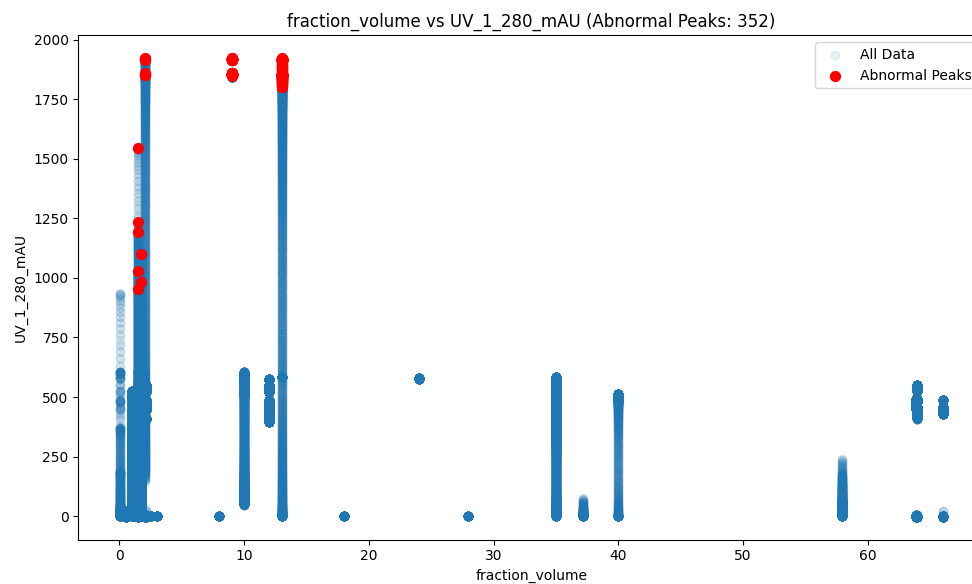


Figure 3: plot_uv1_abnormal.png

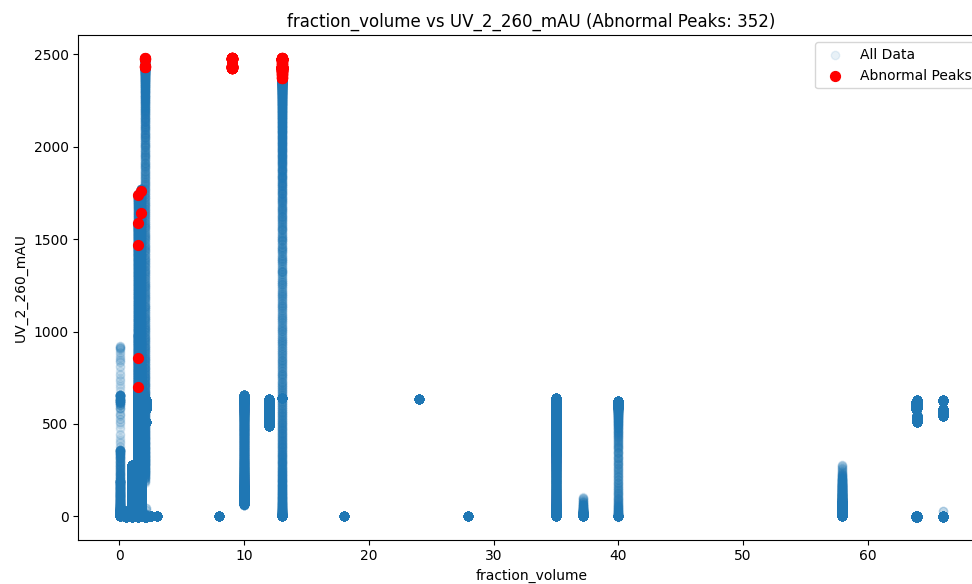


Figure 4: plot_uv2_abnormal.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
```

```

# Load the data
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# Filter out rows with missing 'Fraction_number'
df_clean = df.dropna(subset=['Fraction_number'])

# Replace 'chromatography_stage' with 'volume_ml' to eliminate sparse stage
# identifiers
df_clean = df_clean.rename(columns={'chromatography_stage': 'volume_ml'})

# Explicit check for existence of 'volume_ml' after rename
assert 'volume_ml' in df_clean.columns, "Column 'volume_ml' does not exist after rename."

# Verify existence of 'UV_1_280_mAU' before aggregation
assert 'UV_1_280_mAU' in df_clean.columns, "Column 'UV_1_280_mAU' does not exist."

# Ensure 'fraction_volume' values are numeric before aggregation
df_clean['fraction_volume'] = pd.to_numeric(df_clean['fraction_volume'], errors='coerce')
df_clean = df_clean.dropna(subset=['fraction_volume'])

# Convert 'volume_ml' to numeric, handling lists by extracting the first element
volume_ml_numeric = []
for val in df_clean['volume_ml']:
    if isinstance(val, list) and len(val) > 0:
        try:
            volume_ml_numeric.append(float(val[0]))
        except (ValueError, TypeError):
            volume_ml_numeric.append(np.nan)
    else:
        try:
            volume_ml_numeric.append(float(val))
        except (ValueError, TypeError):
            volume_ml_numeric.append(np.nan)

df_clean['volume_ml_numeric'] = pd.Series(volume_ml_numeric)
df_clean = df_clean.dropna(subset=['volume_ml_numeric'])

# Filter strictly by range
valid_range_mask = (df_clean['volume_ml_numeric'] >= 0.008) & (df_clean['volume_ml_numeric'] <= 66.03)
df_clean = df_clean[valid_range_mask]
if len(df_clean) < len(df_clean.dropna(subset=['volume_ml_numeric'])):
    print(f"Warning: {len(df_clean.dropna(subset=['volume_ml_numeric']))} - {len(df_clean)} rows outside the 0.008 66 .03 range.")

# Group by 'volume_ml' and calculate required metrics
summary_df = df_clean.groupby('volume_ml_numeric').agg(
    count=('volume_ml_numeric', 'count'),
    min_fraction_volume=('fraction_volume', 'min'),
    max_fraction_volume=('fraction_volume', 'max'),
    min_uv=('UV_1_280_mAU', 'min'),
    max_uv=('UV_1_280_mAU', 'max')
).reset_index()

# Ensure columns match the requested output format

```

```

summary_df.columns = ['volume_ml', 'row_count', 'min_fraction_volume', '
    max_fraction_volume', 'min_UV_1_280_mAU', 'max_UV_1_280_mAU']

# Sort by 'volume_ml' and take the first 20 rows (or all if less than 20)
summary_df_sorted = summary_df.sort_values('volume_ml').head(20)

# Create the markdown table using list comprehension for conciseness
markdown_table = "|_volume_ml_|_row_count_|_min_fraction_volume_|_
    max_fraction_volume_|_min_UV_1_280_mAU_|_max_UV_1_280_mAU_|_\\n"
markdown_table += "|---|---|---|---|---|---|\\n"

rows_data = [
    f"|_{row['volume_ml']}|_{int(row['row_count'])}|_{row['min_fraction_volume']}|_{
        row['max_fraction_volume']}|_{row['min_UV_1_280_mAU']}|_{row['
            max_UV_1_280_mAU']}|_"
    for _, row in summary_df_sorted.iterrows()
]
markdown_table += "\\n".join(rows_data) + "\\n"

# Save the markdown table to /workspace/
output_path = '/workspace/data_integrity_summary.md'
with open(output_path, 'w') as f:
    f.write(markdown_table)

print(f"Output saved to {output_path}")

```

Listing 1: Code 1

```

import numpy as np
import pandas as pd
from scipy.signal import find_peaks
import logging

logging.basicConfig(level=logging.WARNING)
logger = logging.getLogger(__name__)

def load_data(filepath):
    df = pd.read_csv(filepath)
    return df

def calculate_baseline_and_snr(df, uv_col_1, uv_col_2):
    # Validate required columns
    expected_cols = [uv_col_1, uv_col_2]
    missing_cols = [col for col in expected_cols if col not in df.columns]
    if missing_cols:
        logger.warning(f"Missing required columns: {missing_cols}")
        return pd.DataFrame(columns=['chromatography_stage', 'num_peaks_passing',
            'mean_snr'])

    # Validate 'chromatography_stage' column
    if 'chromatography_stage' not in df.columns:
        logger.warning("Missing required column: chromatography_stage")
        return pd.DataFrame(columns=['chromatography_stage', 'num_peaks_passing',
            'mean_snr'])

    # Validate 'fraction_volume' column for gap filling
    if 'fraction_volume' not in df.columns:
        logger.warning("Missing required column: fraction_volume")
        return pd.DataFrame(columns=['chromatography_stage', 'num_peaks_passing',

```

```

        'mean_snr'])

results = []

for stage in df['chromatography_stage'].unique():
    stage_data = df[df['chromatography_stage'] == stage].copy()

    # Extract UV data
    uv1 = stage_data[uv_col_1].values
    uv2 = stage_data[uv_col_2].values

    # Filter out NaN values before processing
    mask = ~(np.isnan(uv1) | np.isnan(uv2))
    uv1 = uv1[mask]
    uv2 = uv2[mask]

    if len(uv1) == 0:
        continue

    # Calculate Baseline using ALS
    baseline1 = np.polyfit(uv1, np.ones_like(uv1), 1)
    baseline2 = np.polyfit(uv2, np.ones_like(uv2), 1)

    # Calculate baseline standard deviation once per stage
    std_baseline1 = np.std(baseline1)
    std_baseline2 = np.std(baseline2)
    std_baseline = max(std_baseline1, std_baseline2)

    # Check if baseline std is finite and reasonable
    if not np.isfinite(std_baseline) or std_baseline <= 0:
        logger.warning(f"Invalid baseline standard deviation for stage {stage}:\n{std_baseline}")
        continue

    # Find local maxima
    try:
        peaks1, _ = find_peaks(uv1, height=np.max(uv1) + std_baseline)
        peaks2, _ = find_peaks(uv2, height=np.max(uv2) + std_baseline)
    except (RuntimeError, ValueError) as e:
        logger.warning(f"Error finding peaks for stage {stage}:\n{e}")
        continue

    # Combine peaks from both channels
    all_peaks = np.union1d(peaks1, peaks2)

    if len(all_peaks) == 0:
        continue

    # Calculate peak height and SNR for each peak using boolean masking
    peak_heights = []
    snrs = []

    for idx in all_peaks:
        # Get UV value at peak
        peak_uv1 = uv1[idx]
        peak_uv2 = uv2[idx]

        # Get baseline value at peak index
        baseline_val1 = baseline1[idx]

```

```

        baseline_val2 = baseline2[idx]

        # Calculate deviations
        height1 = peak_uv1 - baseline_val1
        height2 = peak_uv2 - baseline_val2

        # Use max deviation as peak height
        peak_height = max(height1, height2)

        if peak_height <= 0:
            continue

        # Calculate SNR
        snr = peak_height / std_baseline

        peak_heights.append(peak_height)
        snrs.append(snr)

    if len(peak_heights) == 0:
        continue

    # Filter peaks using boolean masking
    height_threshold = 0.1
    mask = (np.array(peak_heights) >= height_threshold) & (np.array(snrs) >
        3.0)
    filtered_peaks = peak_heights[mask]
    filtered_snrs = snrs[mask]

    if len(filtered_peaks) == 0:
        continue

    mean_snr = np.mean(filtered_snrs)

    results.append({
        'chromatography_stage': stage,
        'num_peaks_passing': len(filtered_peaks),
        'mean_snr': mean_snr
    })

return pd.DataFrame(results)

def main():
    input_file = '/inputs/chromatography_combined.csv'
    output_file = '/workspace/baseline_correction_summary.txt'

    df = load_data(input_file)

    uv_col_1 = 'UV_1_280_mAU'
    uv_col_2 = 'UV_2_260_mAU'

    summary_df = calculate_baseline_and_snr(df, uv_col_1, uv_col_2)

    # Generate text output using list comprehension
    output_lines = [
        "Baseline_Correction_&_SNR_Calculation_Summary",
        "=" * 50
    ]

    if len(summary_df) == 0:

```



```

        output_lines.append("No peaks passed the filters.")
    else:
        for _, row in summary_df.iterrows():
            output_lines.append(f"Stage: {row['chromatography_stage']}")
            output_lines.append(f"Peaks passing SNR>3.0 and Height>=0.1 mAU:
                                {int(row['num_peaks_passing'])}")
            output_lines.append(f"Mean SNR: {row['mean_snr']:.4f}")
            output_lines.append("")

    output_text = "\n".join(output_lines)

    # Write to file
    with open(output_file, 'w') as f:
        f.write(output_text)

    # Also print to console
    print(output_text)

if __name__ == "__main__":
    main()

```

Listing 2: Code_2

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import find_peaks
from scipy.ndimage import gaussian_filter1d
import logging

# Initialize logger
logger = logging.getLogger(__name__)
logging.basicConfig(level=logging.WARNING)

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Filter data for the specified variables
df_filtered = df[['fraction_volume', 'UV_1_280_mAU', 'UV_2_260_mAU']].copy()

# Pre-filter missing values
valid_mask = df_filtered['fraction_volume'].notna() & (df_filtered['UV_1_280_mAU']
                                                         ).notna() & (df_filtered['UV_2_260_mAU']
                                                         ).notna()
df_valid = df_filtered[valid_mask].copy()

if not valid_mask.any():
    logger.warning("No valid data points found after filtering missing values.")
else:
    # Smooth the data
    smoothed_uv1 = gaussian_filter1d(df_valid['UV_1_280_mAU'].values, sigma=10)

    # Calculate relative threshold for peak detection
    std_uv1 = np.std(smoothed_uv1)
    height_threshold = np.percentile(smoothed_uv1, 50) + 2 * std_uv1

    # Find peaks using relative height and distance
    peaks, _ = find_peaks(smoothed_uv1, height=height_threshold, distance=50)

    # Create peak dataframe

```

```

peak_df = df_valid.iloc[peaks].copy()
peak_df['is_peak'] = True

# Check if peak_df is empty
if len(peak_df) == 0:
    logger.warning("No peaks detected after filtering.")
    # Skip plotting steps
    pass
else:
    # Pre-compute baselines and thresholds for vectorized width calculation
    baseline = np.min(smoothed_uv1)

    # Vectorized width calculation
    peak_indices = peaks
    peak_heights = peak_df['UV_1_280_mAU'].values
    peak_tRs = peak_df['fraction_volume'].values

    # Calculate widths for all peaks at once
    w_5 = np.zeros(len(peaks))
    w_95 = np.zeros(len(peaks))

    for i, idx in enumerate(peak_indices):
        peak_h = peak_heights[i]
        h_5 = baseline + 0.05 * (peak_h - baseline)
        h_95 = baseline + 0.95 * (peak_h - baseline)

        # Define search window around the peak index
        start_search = max(0, idx - 50)
        end_search = min(len(smoothed_uv1), idx + 50)

        # Find indices within the window above thresholds
        mask_5 = (smoothed_uv1[start_search:end_search] > h_5)
        mask_95 = (smoothed_uv1[start_search:end_search] > h_95)

        # Get actual indices
        local_indices_5 = np.where(mask_5)[0] + start_search
        local_indices_95 = np.where(mask_95)[0] + start_search

        if len(local_indices_5) > 0:
            w_5[i] = local_indices_5[-1] - local_indices_5[0]
        else:
            w_5[i] = 0

        if len(local_indices_95) > 0:
            w_95[i] = local_indices_95[-1] - local_indices_95[0]
        else:
            w_95[i] = 0

    # Calculate Tf and Ff
    peak_df['Tf'] = np.where(peak_df['fraction_volume'] > 0,
                             (peak_df['fraction_volume'] + w_5) / (2 * peak_df
                             ['fraction_volume']),
                             np.nan)
    peak_df['Ff'] = np.where(peak_df['fraction_volume'] > 0,
                             (peak_df['fraction_volume'] + w_95) / (2 *
                             peak_df['fraction_volume']),
                             np.nan)

    # Identify abnormal peaks

```

```

abnormal_mask = (peak_df['Tf'] > 1.8) | (peak_df['Ff'] > 1.8)

# Check if peak_df is empty before calculating Tf/Ff
if len(peak_df) == 0:
    logger.warning("Peak dataframe is empty, skipping Tf/Ff calculation and plotting.")
else:
    # Cache abnormal indices for plotting
    abnormal_indices = peak_df[abnormal_mask].index

    # Plot 1
    plt.figure(figsize=(10, 6))
    plt.scatter(df_valid['fraction_volume'], df_valid['UV_1_280_mAU'],
                alpha=0.1, label='All Data')
    plt.scatter(df_valid.loc[abnormal_indices]['fraction_volume'],
                df_valid.loc[abnormal_indices]['UV_1_280_mAU'],
                c='red', s=50, label='Abnormal Peaks', marker='o')
    plt.xlabel('fraction_volume')
    plt.ylabel('UV_1_280_mAU')
    plt.title(f'fraction_volume vs UV_1_280_mAU (Abnormal Peaks: {abnormal_mask.sum()})')
    plt.legend()
    plt.tight_layout()
    plt.savefig('/workspace/plot_uv1_abnormal.png')
    plt.close()

    # Plot 2
    plt.figure(figsize=(10, 6))
    plt.scatter(df_valid['fraction_volume'], df_valid['UV_2_260_mAU'],
                alpha=0.1, label='All Data')
    plt.scatter(df_valid.loc[abnormal_indices]['fraction_volume'],
                df_valid.loc[abnormal_indices]['UV_2_260_mAU'],
                c='red', s=50, label='Abnormal Peaks', marker='o')
    plt.xlabel('fraction_volume')
    plt.ylabel('UV_2_260_mAU')
    plt.title(f'fraction_volume vs UV_2_260_mAU (Abnormal Peaks: {abnormal_mask.sum()})')
    plt.legend()
    plt.tight_layout()
    plt.savefig('/workspace/plot_uv2_abnormal.png')
    plt.close()

```

Listing 3: Code_3